NUST CHIP DESIGN CENTRE

RISC-V Assembly

Lab Manual # 26
Cache Simulator Design

Name	*Muddassir Ali Siddiqui*
Instructor	*Sir Imran & Sir Umer*
Date	*21st Sep 2025*

1. In-Lab Tasks:

i. Step 01:

```
$ valgrind --log-fd=1 --tool=lackey -v --trace-mem=yes ls -l
```

```
==8010==
==8010== Counted 0 calls to main()
==8010==
==8010== Jccs:
==8010==   total:         180,176
==8010==   taken:         67,939 (38%)
==8010==
==8010== Executed:
==8010==   SBs entered:   191,694
==8010==   SBs completed: 130,732
==8010==   guest instrs:  1,131,534
==8010==   IRStmts:       6,905,930
==8010==
==8010== Ratios:
==8010==   guest instrs : SB entered  = 59 : 10
==8010==        IRStmts : SB entered  = 360 : 10
==8010==        IRStmts : guest instr = 61 : 10
==8010==
==8010== Exit code:       0
[cc@ncdc-0063 cachelab-handout]$
```

ii. Step 02:

```
[cc@ncdc-0063 cachelab-handout]$ make clean && make
rm -rf *.o
rm -f *.tar
rm -f csim
rm -f .csim_results .marker
gcc -g -Wall -Werror -std=c99 -m64 -g -o csim csim.c cachelab.c -lm
[cc@ncdc-0063 cachelab-handout]$ ./test-csim
                        Your simulator     Reference simulator
Points (s,E,b)    Hits  Misses  Evicts    Hits  Misses  Evicts
     3 (1,1,1)       9       8       6       9       8       6 traces/yi2.trace
     6 (4,2,4)       4       5       2       4       5       2 traces/yi.trace
     6 (2,1,4)       2       3       1       2       3       1 traces/dave.trace
     6 (2,1,3)     167      71      67     167      71      67 traces/trans.trace
     6 (2,2,3)     201      37      29     201      37      29 traces/trans.trace
     6 (2,4,3)     212      26      10     212      26      10 traces/trans.trace
     6 (5,1,5)     231       7       0     231       7       0 traces/trans.trace
     9 (5,1,5)  265189   21775   21743  265189   21775   21743 traces/long.trace
    48

TEST_CSIM_RESULTS=48

MAXIMUM POINTS=48
[cc@ncdc-0063 cachelab-handout]$
```

2. Critical Analysis:

The Cache Simulator for Lab 26 was successfully implemented in C++, with all missing code segments completed and verified against the provided reference output. The program accurately models cache operations such as hits, misses, and evictions, confirming correct handling of set indexing, tag comparison, and LRU replacement. Testing showed that the simulator's statistics—hit rate, miss rate, and eviction count—match the expected results, validating the chosen logic and data structures. This exercise strengthened understanding of memory hierarchy concepts and demonstrated how cache parameters like set count, associativity, and block size affect performance. Debugging and validating with the reference implementation reinforced best practices for modular coding and systematic verification, providing practical experience in translating theoretical cache behavior into an efficient, working C++ program.